WEB SYSTEMS AND TECHNOLOGIES

Phase 2 – Backend Development and API Integration

LAB REPORT

Uniformity: An E-Commerce platform for Pre-Loved School and Professional Uniforms

GROUP NAME: TechNic **Course & Block:** BSIT-2A		
MEMBERS	**ROLE**	**SIGNATURE**
Cipriano, Mcxyron B.	*Back-End Developer*	
Hernandez, Kurt Jushua S.	*Database Manager/Tester & Debugger*	
Registrado, Mharie Franz	*Tester & Debugger*	
Red, Paul Orlando B.	*Project, Database, GitHub Manager & Documentation Officer*	
Romano, Jay L.	*Front-End Developer*	

Submitted to:

Guillermo V. Red, Jr., DIT

Assistant Professor IV

I. Overview of Backend Setup

This laboratory report documents the Phase 2 backend development of our project Uniformity, a web-based e-commerce platform for buying and selling pre-loved school and professional uniforms. The backend was built using Node.js and Express, with MongoDB as the database and Mongoose as the Object Data Modeling (ODM) library.

The backend follows a modular and scalable MVC-inspired architecture, separating concerns into distinct folders: models, controllers, routes, and config. This structure ensures maintainability as the project grows across the remaining phases.

Tech Stack Used:

- **Node.js —** JavaScript runtime for the server
- **Express —** Web framework for building RESTful APIs
- **MongoDB —** NoSQL database for storing application data
- **Mongoose —** ODM library for schema definition and database interaction
- **dotenv —** Environment variable management
- **cors —** Cross-Origin Resource Sharing middleware
- **nodemon —** Auto-restart server during development

II. Code Snippets

A. server.js - Main Server Setup

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');

const app = express();
connectDB();

app.use(cors());
app.use(express.json());

app.use('/api/users',      require('./routes/userRoutes'));
```

```
app.use('/api/categories', require('./routes/categoryRoutes'));
app.use('/api/products',   require('./routes/productRoutes'));
app.use('/api/cart',       require('./routes/cartRoutes'));
app.use('/api/orders',     require('./routes/orderRoutes'));

app.listen(process.env.PORT || 5000, () => {
  console.log(`Server running on port ${process.env.PORT ||
5000}`);
});
```

B. Sample Model - Product.js

The Product model defines the schema for uniform listings. It references both the User (as seller) and Category collections using ObjectId foreign keys, reflecting the ERD designed in Phase 1.

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name:        { type: String, required: true },
  size:        { type: String, required: true },
  quantity:    { type: Number, required: true },
  price:       { type: Number, required: true },
  description: { type: String },
  image_url:   { type: String },
  seller_id:   { type: mongoose.Schema.Types.ObjectId, ref: 'User',
required: true },
  category_id: { type: mongoose.Schema.Types.ObjectId, ref:
'Category', required: true },
  created_at:  { type: Date, default: Date.now }
});

module.exports = mongoose.model('Product', productSchema);
```

C. Sample Route - productRoutes.js

Routes are kept thin — they only define the HTTP method and path, then delegate logic to the controller. This separation of concerns makes the code easier to maintain and test.

```
const express = require('express');
const router  = express.Router();
const {
  createProduct, getAllProducts,
  getProductById, updateProduct, deleteProduct
} = require('../controllers/productController');

router.post('/',    createProduct);
router.get('/',     getAllProducts);
router.get('/:id',  getProductById);
router.put('/:id',  updateProduct);
router.delete('/:id', deleteProduct);

module.exports = router;
```

D. Sample Controller - productController.js (partial)

Controllers contain the business logic for each route. The getAllProducts controller supports optional query parameters for filtering by category or seller, enabling the browse and seller dashboard features planned for later phases.

```
exports.getAllProducts = async (req, res) => {
  try {
    const filter = {};
    if (req.query.category) filter.category_id = req.query.category;
    if (req.query.seller)   filter.seller_id   = req.query.seller;
    const products = await Product.find(filter)
      .populate('seller_id',   'name email')
      .populate('category_id', 'name');
    res.json(products);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

III. API Testing

All routes were tested using Thunder Client inside VS Code. Below are the documented test cases for each resource.

A. Users

GET http://localhost:5000/api/users Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

Status: 200 OK Size: 153 Bytes Time: 54 ms

Response Headers 7 Cookies Results Docs


```
[
  {
    "_id": "69da784be9cb0909778f2c76",
    "name": "Juan Dela Cruz",
    "email": "juan@email.com",
    "password": "123456",
    "created_at": "2026-04-11T16:35:23.304Z",
    "__v": 0
  }
]
```


POST http://localhost:5000/api/users Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format


```
{
  "name": "Paul Dela Cruz",
  "email": "paul@email.com",
  "password": "123456"
}
```


Status: 201 Created Size: 151 Bytes Time: 129 ms

Response Headers 7 Cookies Results Docs


```
{
  "name": "Paul Dela Cruz",
  "email": "paul@email.com",
  "password": "123456",
  "_id": "69dbb106714c1ace62413483",
  "created_at": "2026-04-12T14:49:42.038Z",
  "__v": 0
}
```


B. Categories

```
Paw Red@RED MINGW64 ~/Desktop/finalprojectTechNic-BSIT2a/backend (main)
$ node seed.js
[dotenv@17.3.1] injecting env (2) from .env -- tip: ◈ prevent building .env in docker: https://dotenvx.com/prebuild
MongoDB Connected
Categories seeded successfully!
```

GET http://localhost:5000/api/categories Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

Status: 200 OK Size: 141 Bytes Time: 101 ms

Response Headers 7 Cookies Results Docs


```
[
  {
    "_id": "69dbb14da80821891c05897f",
    "name": "School Uniform",
    "__v": 0
  },
  {
    "_id": "69dbb14da80821891c058980",
    "name": "Professional Uniform",
    "__v": 0
  }
]
```

C. Products

POST http://localhost:5000/api/products Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "name": "PE Uniform",
  "size": "Medium",
  "quantity": 3,
  "price": 150,
  "description": "Slightly used PE uniform, good condition",
  "image_url": "https://placeholder.com/image.jpg",
  "seller_id": "69dbb106714c1ace62413483",
  "category_id": "69dbb14da80821891c05897f"
}
```

Status: 201 Created Size: 328 Bytes Time: 47 ms

Response Headers 7 Cookies Results Docs

```
{
  "name": "PE Uniform",
  "size": "Medium",
  "quantity": 3,
  "price": 150,
  "description": "Slightly used PE uniform, good condition",
  "image_url": "https://placeholder.com/image.jpg",
  "seller_id": "69dbb106714c1ace62413483",
  "category_id": "69dbb14da80821891c05897f",
  "_id": "69dbb2e5714c1ace6241348c",
  "created_at": "2026-04-12T14:57:41.830Z",
  "__v": 0
}
```

GET http://localhost:5000/api/products Send

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

parameter value

Status: 200 OK Size: 419 Bytes Time: 98 ms

Response Headers 7 Cookies Results Docs

```
[
  {
    "_id": "69dbb2e5714c1ace6241348c",
    "name": "PE Uniform",
    "size": "Medium",
    "quantity": 3,
    "price": 150,
    "description": "Slightly used PE uniform, good condition",
    "image_url": "https://placeholder.com/image.jpg",
    "seller_id": {
      "_id": "69dbb106714c1ace62413483",
      "name": "Paul Dela Cruz",
      "email": "paul@email.com"
    },
    "category_id": {
      "_id": "69dbb14da80821891c05897f",
      "name": "School Uniform"
    },
    "created_at": "2026-04-12T14:57:41.830Z"
```

D. Cart

GET http://localhost:5000/api/cart/69dbb106714c1ace62413483 Send

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

parameter value

Status: 200 OK Size: 139 Bytes Time: 117 ms

Response Headers 7 Cookies Results Docs

```
{
  "cart": {
    "user_id": "69dbb106714c1ace62413483",
    "_id": "69dbb336714c1ace62413492",
    "created_at": "2026-04-12T14:59:02.774Z",
    "__v": 0
  },
  "items": []
}
```

POST http://localhost:5000/api/cart/add Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "cart_id": "69dbb336714c1ace62413492",
  "product_id": "69dbb2e5714c1ace6241348c",
  "quantity": 1
}
```

Status: 201 Created Size: 132 Bytes Time: 45 ms

Response Headers 7 Cookies Results Docs

```
{
  "cart_id": "69dbb336714c1ace62413492",
  "product_id": "69dbb2e5714c1ace6241348c",
  "quantity": 1,
  "_id": "69dbb413714c1ace6241349a",
  "__v": 0
}
```

E. Orders

POST http://localhost:5000/api/orders Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "buyer_id": "69dbb106714c1ace62413483",
  "delivery_address": "123 Masbate St., Bicol Region",
  "total_amount": 150,
  "items": [
    {
      "product_id": "69dbb2e5714c1ace6241348c",
      "seller_id": "69dbb106714c1ace62413483",
      "quantity": 1,
      "price": 150
    }
  ]
}
```

Status: 201 Created Size: 378 Bytes Time: 270 ms

Response Headers 7 Cookies Results Docs

```
{
  "order": {
    "buyer_id": "69dbb106714c1ace62413483",
    "delivery_address": "123 Masbate St., Bicol Region",
    "total_amount": 150,
    "status": "pending",
    "_id": "69dbb47e714c1ace6241349d",
    "created_at": "2026-04-12T15:04:30.996Z",
    "__v": 0
  },
  "orderItems": [
    {
      "product_id": "69dbb2e5714c1ace6241348c",
      "seller_id": "69dbb106714c1ace62413483",
      "quantity": 1,
      "price": 150,
      "order_id": "69dbb47e714c1ace6241349d"
    }
  ]
```

PATCH http://localhost:5000/api/orders/item/69dbb87eb9408d984fba0531/ful Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

Status: 200 OK Size: 28 Bytes Time: 374 ms

Response Headers 7 Cookies Results Docs

```
{
  "message": "Item fulfilled"
}
```

PATCH http://localhost:5000/api/orders/69d8f4810a10e5b9c1ecf59c/deliver Send

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

Status: 200 OK Size: 39 Bytes Time: 46 ms

Response Headers 7 Cookies Results Docs

```
{
  "message": "Order marked as delivered"
}
```

IV. Backend Folder Structure

- backend
  - config
  - controllers
    - cartController.js
    - categoryController.js
    - orderController.js
    - productController.js
    - userController.js
  - models
    - Cart.js
    - CartItem.js
    - Category.js
    - Order.js
    - OrderItem.js
    - Product.js
    - User.js

- backend
  - models
    - OrderItem.js
    - Product.js
    - User.js
  - routes
    - authRoutes.js
    - cartRoutes.js
    - categoryRoutes.js
    - orderRoutes.js
    - productRoutes.js
    - userRoutes.js
  - .env
  - package-lock.json
  - package.json
  - README.md
  - seed.js
  - server.js
  - database
    - .gitkeep

IV. Group Member Contributions

Member	Role	Contribution
Cipriano, Mcxyron B.	Back-End Developer	Developed all Express routes and controllers. Built the order fulfillment logic including the auto-update of ORDERS status when all ORDER_ITEMS are fulfilled.
Hernandez, Kurt Jushua S.	Database Manager / Tester & Debugger	Designed and implemented all Mongoose models based on the finalized updated ERD. Set up the MongoDB connection via Mongoose and managed the database configuration.
Registrado, Mharie Franz	Tester & Debugger	Conducted API testing using Thunder Client for all routes. Documented test cases, identified and reported bugs, and also helps in making this lab report in this Phase.
Red, Paul Orlando B.	Project, Database, GitHub Manager & Documentation Officer	Managed the GitHub repository structure and commit history. Wrote the backend README.md and this lab report. Coordinated task assignments across the group.
Romano, Jay L.	Front-End Developer	Assisted in reviewing the API endpoints and response structures to ensure alignment with the planned frontend UI for Phase 3.

Prepared by:

Mcxyron B. Cipriano
Back-End Developer

Kurt Jushua S. Hernandez
Database Manager/Tester & Debugger

Mharie Franz Registrado
Tester & Debugger

Paul Orlando B. Red
Project, Database, GitHub Manager & Documentation Officer

Jay L. Romano
Front-End Developer